Improved Prompts Documentation

Final prompts for all three tasks, the reasoning behind each improvement, and notes on which prompt engineering techniques worked best per task type.

1. Sentiment Analysis

Final Prompt

```
Classify the sentiment of a customer message as a percentage breakdown across three
categories: Positive, Negative, Neutral. The three percentages must add up to 100.
Respond with only the breakdown in this exact format, no explanation: Positive: X%, Negative:
X%, Neutral: X%

Examples:
Message: "This is the worst purchase I've ever made, total waste of money."
Sentiment: Positive: 0%, Negative: 95%, Neutral: 5%

Message: "It arrived on time and works as described."
Sentiment: Positive: 20%, Negative: 5%, Neutral: 75%

Message: "Absolutely love it, best decision I've made this year!"
Sentiment: Positive: 95%, Negative: 0%, Neutral: 5%

Now classify this message:
Message: "I love this product! It's exactly what I needed."
Sentiment:
```

Why This Improvement Was Made

The baseline prompt (v1) reached only 67% consistency after 15 runs because the model was free to answer in different ways. Small wording changes and one formatting difference reduced consistency. Version 2 fixed this by forcing one exact output format, giving 100% consistency. Version 3 added few-shot examples, making the format even more stable. In the final version, I replaced the single label with a percentage breakdown. This keeps the output structured while showing mixed sentiment more clearly.

Technique That Worked Best

The best combination was a clear output format together with few-shot examples. This improved consistency from 67% to 100% because the model no longer had freedom to change the wording.

2. Data Extraction

Final Prompt

```
Extract structured information from the customer feedback below.

Customer feedback: "I ordered item #12345 on March 15th. The delivery was fast but the
packaging was damaged."

Think through this step by step before answering:
1. Identify the order/item number mentioned.
2. Identify the date the order was placed.
3. Determine how the delivery speed is described. Choose the closest match: fast, on-time,
delayed, average, slow, or very slow.
4. Determine the condition of the packaging. Choose the closest match: damaged, intact,
opened, tampered, crushed, wet, or missing.

Final JSON:
{
  "order_number": "...",
  "order_date": "...",
  "delivery_speed": "...",
  "packaging_condition": "..."
}
```

Why This Improvement Was Made

The baseline prompt (v1) was already 100% consistent for this test case, but it did not explain its reasoning and had no strict output format. Version 2 added a JSON structure, making the output easy to parse. Version 3 added step-by-step reasoning before the JSON. The final version kept that approach and expanded the category options, allowing the model to choose more suitable values for different customer feedback.

Technique That Worked Best

Chain-of-thought reasoning together with a clear JSON schema worked best. The output stayed consistent and was easier to understand. The only downside is that the reasoning must be removed before parsing the JSON, and the final version also adds code fences that need to be stripped.

3. Product Description

Final Prompt

```
Write a product description for a wireless mouse that costs $29.99.

Tone: Premium, refined, understated, quality-focused language.

Requirements:
- Use one bold headline
- Write one introduction with 2-3 sentences
- Add a "Key Features:" section with exactly 4 bullet points
- End with one sentence that mentions the price

Keep the response between 120 and 150 words.
Each bullet point should be one sentence and no more than 20 words.

Match the tone above throughout.
Do not use words like "perfect," "game-changing," "unbeatable," or "revolutionary."
Do not ask questions or use exclamation marks.

Example (different product, showing the exact format, length, and tone to match):

Reliable Comfort for Every Workday

This ergonomic keyboard is built for long, focused workdays, combining a cushioned wrist rest
with responsive, whisper-quiet keys that hold up under heavy daily typing. Its low-profile
design fits naturally under your hands, reducing strain whether you're drafting reports or
replying to messages back to back.

Key Features:
- A cushioned wrist rest reduces strain and keeps your hands comfortable through long typing
sessions.
- Responsive, whisper-quiet keys deliver a smooth, tactile feel without disturbing coworkers
nearby at your desk.
- A stable wireless connection holds steady across typical desk and office distances without
dropouts.
- The compact, low-profile design fits easily into any workspace, from a home office to a
shared desk.

Upgrade your setup with this ergonomic keyboard for just $49.99.

Now write the description for the wireless mouse, matching this exact structure, length, and
style.
```

Why This Improvement Was Made

The baseline prompt (v1) had almost no constraints, so the model changed the product name, battery life, and other details in almost every run. Version 2 added structure and formatting rules, making the layout much more consistent. Version 3 added a worked example, which improved the word count and stabilized the overall structure. The final version added a premium tone to make the writing sound more polished.

Technique That Worked Best

A worked few-shot example together with clear structure and length requirements gave the biggest improvement. It made the headline and overall format much more consistent. Adding the premium tone improved the writing style and word count, but it also reduced consistency slightly. This shows that improving one aspect of a prompt can sometimes make another aspect a little less stable.

Summary: Techniques That Worked Best by Task Type

Classification tasks work best with a fixed output format and a few examples. Structured extraction tasks benefit from step-by-step reasoning and a clear JSON schema, although the JSON needs to be extracted before parsing. Open-ended writing tasks work best with an example plus clear structure and length requirements. Tone instructions help the writing style, but they can reduce consistency slightly.

Structured extraction tasks respond best to chain-of-thought reasoning plus an explicit schema with enumerated category options, at the cost of needing a parsing step to strip reasoning text and any code fences before the JSON can be loaded directly.

Open-ended generation tasks (product description) respond best to a worked few-shot example paired with explicit structure and length constraints; this is what fixes identity and structural instability. Style controls like tone specification are a separate, useful axis, but should be expected to trade off slightly against the consistency gained from structure alone.